경력기술서

기업 프로젝트

아래 내용은 실제 수행한 회사 프로젝트를 중심으로 정리했으며, 회사 소스는 사내 GitLab에 있어 공개하지 않습니다.

청오엔에프(도미노피자) | 프론트엔드 개발자 | 2023.03 - 2026.03

- **신규 플랫폼 개발 (정식 프로젝트, 2024.06 ~ 2026.03)**

  기술 스택 | Next.js · TypeScript · Redux · TanStack Query · WebSocket(STOMP) · Turborepo(Monorepo)

  도미노피자 신규 웹 플랫폼에서 회원·인증, 메뉴 목록·상세, 장바구니·주문 흐름과 실시간 주문 상태 UI를 개발했습니다. 서비스 도메인별 구조와 디자인 시스템을 기반으로 비즈니스 로직을 페이지에 통합하고, 사용자 흐름이 끊기는 지점을 중심으로 개선했습니다.

  주요 업무

  - **메뉴 옵션 유지 및 로그인 후 주문 흐름 복원**

    문제

    비로그인 사용자가 메뉴 옵션을 선택한 뒤 장바구니 담기 또는 즉시 주문을 시도하면 인증 페이지로 이동하고, 로그인 후 기존 선택 상태가 초기화되어 다시 선택해야 하는 문제가 있었습니다.

    구현 / 개선

    - redux-persist를 활용해 비회원 상태에서도 메뉴 선택 데이터를 Local Storage에 유지
    - redirect/return URL을 활용해 인증 완료 후 원래 사용자 흐름으로 복귀
    - 로그인 후 최신 메뉴·품절 상태를 기준으로 저장된 선택값을 검증하고 유효한 옵션 복원
    - 복원된 선택 상태를 기반으로 로그인 전 요청한 장바구니 담기 또는 즉시 주문 동작을 이어서 수행하고, 실패 시 알림 제공
    - 수량 제한, 품절 상태 및 옵션 간 의존관계에 따른 복합 선택 제약 로직 구현

    결과

    - 로그인으로 중단된 메뉴 선택 흐름을 복원하여 사용자의 반복 선택을 줄이고 주문 흐름 유지

  - **회원가입·인증 플로우 및 복합 폼 검증**

    문제

    회원가입 과정에서 입력값 검증, 약관 동의, 본인인증, 인증 이후 화면 복귀 등 여러 단계가 연결되어 있어 일관된 검증과 사용자 흐름 관리가 필요했습니다.

    구현 / 개선

    - React Hook Form + Zod 조합으로 필드별·조건별 복합 유효성 검증 체계 구성
    - KG이니시스 본인인증을 회원가입 플로우에 연동
    - 약관 동의와 필수 입력 조건을 검증하고 단계별 상태를 UI에 반영
    - redirect URL을 쿠키 기반으로 관리해 로그인/인증 후 사용자가 진행하던 위치로 복귀하도록 처리

    결과

    - 회원가입과 인증 과정의 조건을 한 흐름으로 연결해 입력 오류와 인증 전후의 화면 단절 감소

  - **도메인별 비즈니스 로직 모듈화 및 UI 페이지 통합**

    문제

    MVP에서는 빠른 기능 검증을 우선하면서 일부 UI 컴포넌트에 비즈니스 로직이 함께 구성되어 있었습니다. 정식 플랫폼에서는 기능 확장과 유지보수를 고려해 UI와 도메인 로직의 역할을 보다 명확하게 분리할 필요가 있었습니다.

    구현 / 개선

    - Turborepo 기반 모노레포 아키텍처 및 도메인·역할별 프로젝트 구조 설계 논의에 참여
    - 인증·메뉴·장바구니·주문 등 도메인 단위로 비즈니스 로직을 별도의 커스텀 훅으로 분리하고 UI에서 필요한 형태로 결합
    - 컴포넌트에서 직접 비즈니스 로직을 처리하는 방식 대신, 훅을 통해 필요한 데이터를 주입받도록 변경

- Storybook 기반 디자인 시스템 컴포넌트를 활용해 화면 일관성을 유지하면서 도메인 로직을 통합

결과

- UI와 비즈니스 로직의 역할을 분리해 도메인 로직을 재사용할 수 있는 구조로 구성하고, 기능 변경 시 영향 범위를 줄일 수 있도록 개선

- **MVP 프로젝트 (2023.12 ~ 2024.05)**

  기술 스택 | Next.js · TypeScript · Redux · React Query · WebSocket(STOMP)

  정식 신규 플랫폼 착수 전 핵심 사용자 흐름과 기술 구성을 빠르게 검증하기 위한 MVP를 개발했습니다. 프로토타입을 시작으로 팀 단위 MVP 개발로 확장되었으며, 이후 정식 신규 플랫폼 프로젝트로 이어지는 기반이 되었습니다.

  주요 업무

  - **Next.js 기반 메뉴 페이지 구축 및 ISR 렌더링 적용**

    문제

    기존 운영 사이트는 서버 중심의 페이지 단위 요청·응답 구조로, 메뉴 탐색 과정에서 전체 페이지가 다시 로드되는 등 사용자 흐름이 끊기는 구간이 있었습니다. MVP에서는 이러한 사용자 경험을 개선하기 위해 React/Next.js 기반으로 메뉴 페이지를 구축했으며, 특히 메뉴 목록·상세는 사용자 접근 빈도에 비해 데이터 변경 빈도가 낮아 빠른 페이지 제공과 주기적인 데이터 갱신을 함께 고려한 렌더링 전략이 필요했습니다.

    구현 / 개선

    - React/Next.js 기반으로 MVP 전반의 사용자 화면을 구축하고, 상태 변화에 필요한 UI만 업데이트되는 구조로 구현
    - Next.js ISR을 메뉴 목록·상세 페이지에 적용해 정적 생성된 페이지를 제공하고, 일정 주기로 변경되는 메뉴 데이터가 반영되도록 구성

    결과

    - 기존 페이지 단위 요청·응답 방식에서 발생하던 불필요한 전체 페이지 전환을 줄이고 사용자 인터랙션 흐름 개선
    - 메뉴 목록·상세는 정적 렌더링 기반으로 제공하면서 변경된 메뉴 데이터를 주기적으로 반영할 수 있는 구조 구성

  - **WebSocket 기반 실시간 주문 상태 동기화**

    문제

    운영 중인 웹사이트에서는 주문 상태 변경이 실시간으로 반영되지 않아 사용자가 새로고침해야만 최신 정보를 확인할 수 있었으며, 이를 사용자 불편 사항으로 인식했습니다.

    구현 / 개선

    - STOMP 기반 WebSocket으로 주문 상태 변경 메시지 수신
    - 수신한 주문 상태를 주문 ID 기준으로 Redux에 반영하고 관련 UI가 즉시 갱신되도록 구성

    결과

    - 주문 상태 변경을 실시간으로 UI에 반영하여 별도 새로고침 없이 최신 주문 상태를 확인할 수 있도록 개선
    - MVP에서 검증한 WebSocket 기반 주문 상태 처리 방식을 정식 신규 플랫폼의 실시간 주문 상태 UI로 확장 적용

  - **장바구니 관리 및 체크아웃 기능 최적화**

    문제

    장바구니는 서버에 저장되는 사용자 데이터이기 때문에 PC·모바일 등 여러 환경에서 변경될 수 있었고, 클라이언트에 캐시된 데이터만 유지할 경우 다른 디바이스에서 변경된 최신 상태를 즉시 확인하기 어려웠습니다.

    구현 / 개선

    - React Query 기반 장바구니 서버 상태 관리
    - refetchInterval을 적용해 일정 주기로 최신 장바구니 데이터를 조회하고 다중 디바이스 간 상태 동기화

    결과

    - PC·모바일 등 서로 다른 환경에서도 서버의 최신 장바구니 상태를 주기적으로 반영하도록 구성

  - **결제 및 체크아웃 흐름 구현**

    문제

주문 서비스의 핵심인 결제 수단별 호출 방식과 완료 처리 흐름을 하나의 사용자 경험 안에서 안정적으로 연결해야 했습니다.

구현 / 개선

- 네이버페이, 카카오페이, 애플페이, KG이니시스 등 결제 서비스 연동
- 결제창 호출부터 성공/완료 이후 처리까지 프론트엔드 플로우 구현

결과

- 복수 결제수단을 실제 주문 흐름에 연결해 MVP에서 결제까지 이어지는 핵심 사용자 여정 검증

- **웹 성능 최적화 및 Lighthouse 점수 개선**

  문제

  기존 운영 사이트는 페이지 로딩과 사용자 인터랙션 측면에서 개선이 필요한 구간이 있었으며, Lighthouse 측정에서도 성능 지표가 60~70점대로 확인되었습니다. 이를 참고해 MVP에서는 기능 구현뿐 아니라 성능·접근성·SEO 등 웹 품질을 함께 고려할 필요가 있었습니다.

  구현 / 개선

  - Next.js Image 컴포넌트를 적용하고 시맨틱 HTML 및 ARIA 속성을 활용해 성능·접근성 개선
  - Lighthouse를 활용해 성능·접근성·SEO 등 주요 지표 점검

  결과

  - MVP의 Lighthouse 성능·접근성·SEO 등 주요 지표 90점 이상 달성

게임인스 | 웹퍼블리셔 | 2021.09 - 2022.05

기술 스택 | HTML5 · CSS3 · JavaScript · jQuery · Bootstrap · Figma

주요 업무

- **커뮤니티 웹사이트 퍼블리싱 및 실시간 문의 UX 개선**

  문제

  실시간 문의 기능에서 소요 시간, 취소 방법 등 동일한 질문이 반복되어 상담자가 매번 직접 답변해야 했습니다.

  구현 / 개선

  - 웹사이트 퍼블리싱 및 유지보수, 기기·브라우저 호환성 이슈 대응
  - 반복 문의를 사전 정의된 답변으로 확인할 수 있는 선택형 UI 제안 및 구현
  - jQuery로 버튼 선택 시 사전 정의된 답변이 즉시 표시되도록 인터랙션 구현
  - IE 환경에서도 원활한 UI 제공을 위해 CSS Grid 대신 CSS Flexbox로 대체 적용
  - 브라우저 벤더 접두사(Prefix)를 활용하여 CSS 호환성을 유지하고 다양한 브라우저에서 UI가 일관되게 보이도록 개선

  결과

  - 반복 문의 감소 및 상담 응대 효율 개선
  - IE 포함 다양한 브라우저에서 UI가 정상적으로 유지되며, 크로스 브라우징 문제 해결

개인 프로젝트

React + HLS Lecture Player | 개인 프로젝트 | 2026.09

기술 스택 | Vite · React · TypeScript · hls.js · Tailwind CSS

온라인 강의 플랫폼의 핵심인 영상 재생 경험을 직접 이해하고 구현하기 위해 HLS 기반 강의 플레이어를 제작했습니다. 회원·결제·백엔드 기능을 확장하지 않고 video/HLS lifecycle, 재생 안정화, 이어보기와 반응형 수강 UX에 범위를 집중했습니다.

- HTML5 video element와 ref를 연결하고 play/pause/timeupdate/loadedmetadata/waiting/playing/ended 이벤트를 기반으로 재생 상태 관리
- Safari 등 native HLS 지원 브라우저와 Chrome/Edge의 hls.js 재생 방식을 분기하고 .m3u8 source 연결
- HLS instance 생성·source load·video attach·event 등록을 hook으로 분리하고 source 변경/unmount 시 hls.destroy() cleanup 적용

- idle/loading/playing/buffering/error/ended 상태를 정의하고 buffering overlay, 오류 메시지, retry 흐름 구현
- currentTime을 강의별로 localStorage에 저장하고 metadata 로딩 이후 마지막 시청 위치에서 이어보기 제공
- 강의별 진도율과 완료 상태를 저장하고 강의 전환, 0.75x~2.0x 배속, 현재 재생 시간 표시 구현
- 375/768/1024/1440px 환경에서 반응형 UI를 점검하고 16:9 영상 비율 유지
- 잘못된 m3u8 URL, 느린 네트워크, 빠른 source 전환, 새로고침·브라우저 재실행을 직접 재현해 error/buffering/cleanup/복원 동작 검증

프로젝트에서 확인한 핵심 이해

HLS의 manifest(.m3u8)와 segment 요청 흐름, native HLS와 hls.js의 역할 차이, source 변경 시 lifecycle 정리의 필요성, 네트워크 상태가 buffering/error UX에 미치는 영향, localStorage 기반 시청 위치 복원의 한계와 서버 동기화 확장 지점을 설명할 수 있도록 구현 범위를 구성했습니다.

인스타그램 클론 | 개인 프로젝트 | 2024.05 - 2024.08

기술 스택 | Next.js · TypeScript · Firebase Authentication · TanStack Query · Zustand · MSW · react-virtual

- Firebase 이메일/OAuth 인증과 피드·댓글 중심의 SNS 기능 구현
- useInfiniteQuery 기반 무한 스크롤과 react-virtual·ResizeObserver를 활용한 동적 피드 가상화 렌더링 구현
- 댓글 스레드, 사용자 태그·자동완성, 검색, 다크모드, 스켈레톤·빈 상태 UI 구현
- Next.js pages/api 기반 서버리스 API와 Vercel 배포 구성

MOMO | 6인 팀 프로젝트 | 2022.12 - 2023.02

기술 스택 | Next.js · Recoil · Styled-components · TossPayments API

- 모임·멘토링 예약 웹 애플리케이션 개발
- Recoil을 활용한 전역 상태 관리와 조건부 UI 구성
- TossPayments API를 연동해 결제창 호출 및 결제 성공 이후 예약 처리 흐름 구현

- 웹 퍼블리싱 프로젝트 : Icyeong

기술 경험

- **프론트엔드: React, Next.js, Vue.js**

  → 대부분의 프로젝트를 React 및 Next.js 기반으로 진행하며, SSR·SSG·ISR 등 렌더링 방식을 프로젝트 요구사항에 따라 적용한 경험

  → MVP 이전 단계에서 Nuxt.js, Pinia, Vite를 활용해 빠른 프로토타이핑 및 상태 관리 적용 경험
- **상태 관리: Redux Toolkit, Recoil, Zustand, Tanstack Query**

  → 프로젝트별 요구사항에 따라 Redux Toolkit, Recoil, Zustand을 적절히 활용하여 상태 관리 구현

  → TanStack Query 기반 서버 상태 관리·캐싱·무한 스크롤 및 react-virtual 기반 가상화 렌더링 경험
- **스타일링: Styled-Components, CSS, SCSS, Tailwind CSS**

  → CSS-in-JS 방식과 전통적인 CSS, SCSS를 유연하게 적용하여 프로젝트 스타일링 경험 보유
- **네트워킹: REST API, WebSocket (STOMP)**

  → REST API를 활용한 데이터 통신 경험

  → WebSocket(STOMP) 기반 연결·구독·메시지 수신 및 Redux와 연계한 실시간 UI 상태 반영 경험
- **인증: OAuth, JWT**

  → OAuth를 활용한 카카오, 구글, 네이버 소셜 로그인 연동 경험

  → JWT를 활용한 인증 토큰 관리 및 사용자 세션 유지 경험
- **타입스크립트:**

  → Next.js 프로젝트를 포함한 모든 프로젝트에서 TypeScript를 적용하여 개발

  → 컴포넌트 props와 API 응답 타입을 정의하고 Type Narrowing을 활용해 타입 안정성 향상

- **배포: Vercel, Github**

  → Vercel을 활용한 Next.js 및 React 프로젝트 배포 경험
- **디자인 협업:** Figma

  → UI/UX 디자인 협업을 위한 Figma 활용 가능.